ML Experiment Number 9

Student Name : Sneha Gopal Bhalekar

Batch : cs1

Roll Number : cs05

Aim

To implement Classification and Regression Trees (CART) using Python on the Palmer Penguins Dataset Extended.

Objectives

1. To understand the concept and working of the CART algorithm.
2. To implement Decision Tree Classification using Python.
3. To visualize the constructed Decision Tree structure and decision boundaries.
4. To analyze impurity measures such as Gini Index and Entropy.
5. To compare the decision boundary of a Decision Tree with a linear classification boundary (e.g., Logistic Regression).
6. To evaluate and compare the performance of both classifiers.

Relevant Theory

###Classification and Regression Trees (CART) CART is a non-parametric supervised machine learning algorithm used for both classification and regression tasks. It builds a binary decision tree by recursively splitting the dataset based on feature values that maximize information gain or minimize impurity.

A tree consists of:

- Root Node: The topmost node representing the entire dataset.
- Internal Nodes: Decision points based on feature splits.
- Leaf/Terminal Nodes: Output class labels or predictions.

Impurity MeasuresGini Index (Gini Impurity): Measures the frequency with which a randomly chosen element from the set would be incorrectly labeled.

$$Gini(D) = 1 - \sum_{i=1}^{k} p_i^2$$

where p_i is the probability of an item belonging to class i.Entropy & Information Gain: Measures the amount of impurity or randomness in the data.

$$Entropy(D) = -\sum_{i=1}^{k} p_i \log_2(p_i)$$

Dataset InformationGet the Palmer Penguins Dataset Extended from Kaggle using the following link:https://www.kaggle.com/datasets/samybaladram/palmers-penguin-dataset-extended The Palmer Penguins dataset contains measurements of penguin species across islands in the Palmer Archipelago:AdelieChinstrapGentooThe primary numerical features are:culmen_length_mm (or bill_length_mm)culmen_depth_mm (or bill_depth_mm)flipper_length_mmbody_mass_gFor visualization, we will use: culmen_length_mm and flipper_length_mm.

Task 1: Import Libraries and Load Dataset

- Load essential libraries (numpy, pandas, matplotlib, seaborn, sklearn).
- Load the Palmer Penguins Extended dataset.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay
)

# Load the standard Palmer Penguins dataset
url = "/content/palmerpenguins_extended.csv"
df = pd.read_csv(url)

print("Dataset loaded successfully!")
print(df.head())
print("\nDataset shape:", df.shape)
print("\nColumn names:", df.columns.tolist())
```

```
Dataset loaded successfully!
  species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Biscoe            53.4           17.8              219.0
1  Adelie  Biscoe            49.3           18.1              245.0
2  Adelie  Biscoe            55.7           16.6              226.0
```

```
3  Adelie  Biscoe            38.0           15.6              221.0
4  Adelie  Biscoe            60.7           17.9              177.0

   body_mass_g     sex  diet life_stage health_metrics  year
0       5687.0  female  fish      adult     overweight  2021
1       6811.0  female  fish      adult     overweight  2021
2       5388.0  female  fish      adult     overweight  2021
3       6262.0  female  fish      adult     overweight  2021
4       4811.0  female  fish   juvenile     overweight  2021

Dataset shape: (3430, 11)

Column names: ['species', 'island', 'bill_length_mm', 'bill_depth_mm',
'flipper_length_mm', 'body_mass_g', 'sex', 'diet', 'life_stage',
'health_metrics', 'year']
```

Task 2: Select Features and Target & Preprocess

- Handle missing values (e.g., drop rows with missing values or impute with median).
- Select features (X) such as culmen_length_mm, culmen_depth_mm, flipper_length_mm, body_mass_g and set species as the target label (y).
- Encode categorical targets if necessary using LabelEncoder.

```
# Rename columns to consistent names
df = df.rename(columns={
    "bill_length_mm": "culmen_length_mm",
    "bill_depth_mm": "culmen_depth_mm"
})

features = [
    "culmen_length_mm",
    "culmen_depth_mm",
    "flipper_length_mm",
    "body_mass_g"
]

target = "species"

# Keep required columns and remove missing values
df = df[features + [target]].dropna().copy()

# Encode species labels into numbers
le = LabelEncoder()
df["species_encoded"] = le.fit_transform(df[target])

X = df[features]
```

```
y = df["species_encoded"]

print("Missing values after preprocessing:")
print(X.isnull().sum())

print("\nSpecies classes:", list(le.classes_))
print("\nNumber of records:", len(df))
print(df.head())
```

```
Missing values after preprocessing:
culmen_length_mm     0
culmen_depth_mm      0
flipper_length_mm    0
body_mass_g          0
dtype: int64

Species classes: ['Adelie', 'Chinstrap', 'Gentoo']

Number of records: 3430
   culmen_length_mm  culmen_depth_mm  flipper_length_mm  body_mass_g
species  \
0              53.4             17.8              219.0       5687.0
Adelie
1              49.3             18.1              245.0       6811.0
Adelie
2              55.7             16.6              226.0       5388.0
Adelie
3              38.0             15.6              221.0       6262.0
Adelie
4              60.7             17.9              177.0       4811.0
Adelie

   species_encoded
0                0
1                0
2                0
3                0
4                0
```

Task 3: Split the Data

- Divide the dataset into training and testing sets using train_test_split.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

```
)

print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

```
Training records: 2744
Testing records: 686
```

Task 4: Implement CART Classifier

- Create a DecisionTreeClassifier model using the Gini criterion (or Entropy).
- Fit the model on the training data and make predictions on the test set.

```
cart_model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

cart_model.fit(X_train, y_train)

y_pred_cart = cart_model.predict(X_test)

print("CART model trained successfully!")
```

```
CART model trained successfully!
```

Task 5: Evaluate CART Model

- Accuracy: Calculate classification accuracy.
- Confusion Matrix: Generate the matrix to analyze misclassifications across species.
- Classification Report: Output Precision, Recall, F1-Score, and Support.

```
cart_accuracy = accuracy_score(y_test, y_pred_cart)

print("CART Accuracy:", round(cart_accuracy * 100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_cart))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred_cart,
    labels=np.arange(len(le.classes_)),
```

```
    target_names=le.classes_,
    zero_division=0
))

ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred_cart,
    labels=np.arange(len(le.classes_)),
    display_labels=le.classes_,
    cmap="Blues"
)
plt.title("CART Confusion Matrix")
plt.show()
```

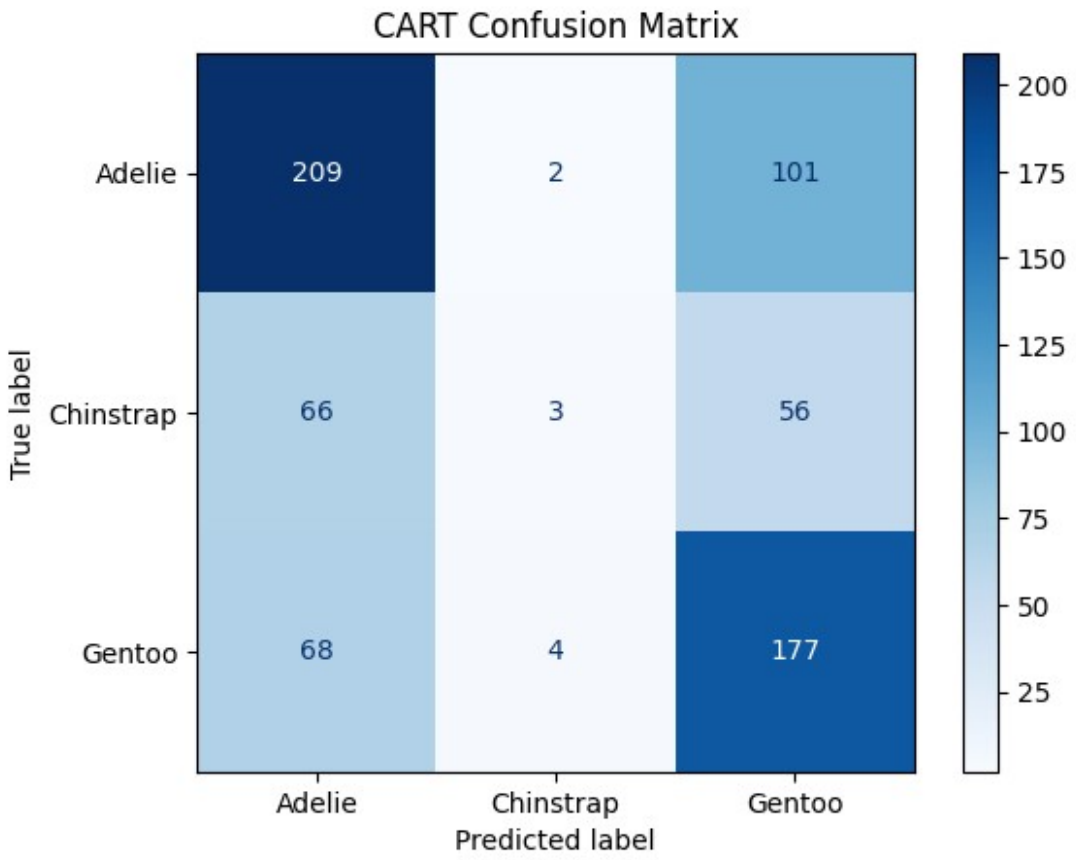
```
CART Accuracy: 56.71 %

Confusion Matrix:
[[209   2 101]
 [ 66   3  56]
 [ 68   4 177]]

Classification Report:
              precision    recall  f1-score   support

      Adelie       0.61      0.67      0.64       312
   Chinstrap       0.33      0.02      0.04       125
      Gentoo       0.53      0.71      0.61       249

    accuracy                           0.57       686
   macro avg       0.49      0.47      0.43       686
weighted avg       0.53      0.57      0.52       686
```



Task 6: Visualize Decision Tree & Decision Boundary

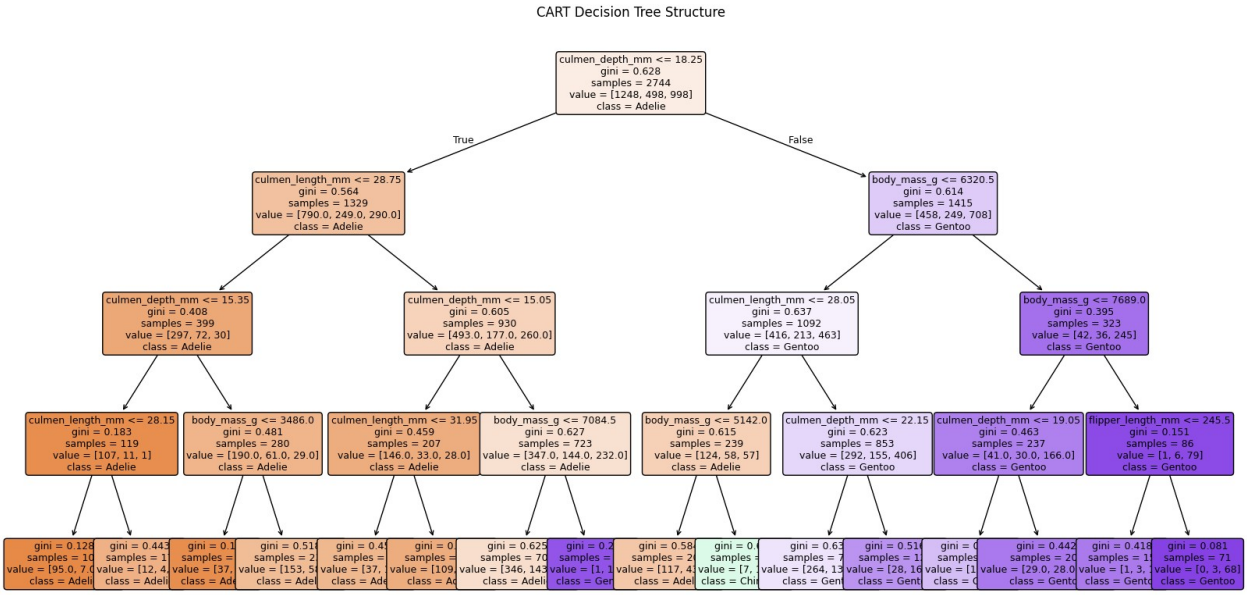
- Visualize Tree Structure: Plot the decision tree diagram using sklearn.tree.plot_tree.
- Select 2-D Features: Select culmen_length_mm and flipper_length_mm.

- Train 2-D Model & Plot Boundary: Train the decision tree on 2D data and plot axis-aligned rectangular decision boundaries along with data points

```
plt.figure(figsize=(20, 10))

plot_tree(
    cart_model,
    feature_names=features,
    class_names=le.classes_,
    filled=True,
    rounded=True,
    fontsize=9
)
```

```
plt.title("CART Decision Tree Structure")
plt.show()
```




```
X2 = df[["culmen_length_mm", "flipper_length_mm"]]
y2 = df["species_encoded"]

X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2,
    test_size=0.2,
    random_state=42,
    stratify=y2
)

cart_2d = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

cart_2d.fit(X2_train, y2_train)

# Create a grid for the decision boundary
x_min, x_max = X2.iloc[:, 0].min() - 2, X2.iloc[:, 0].max() + 2
y_min, y_max = X2.iloc[:, 1].min() - 5, X2.iloc[:, 1].max() + 5

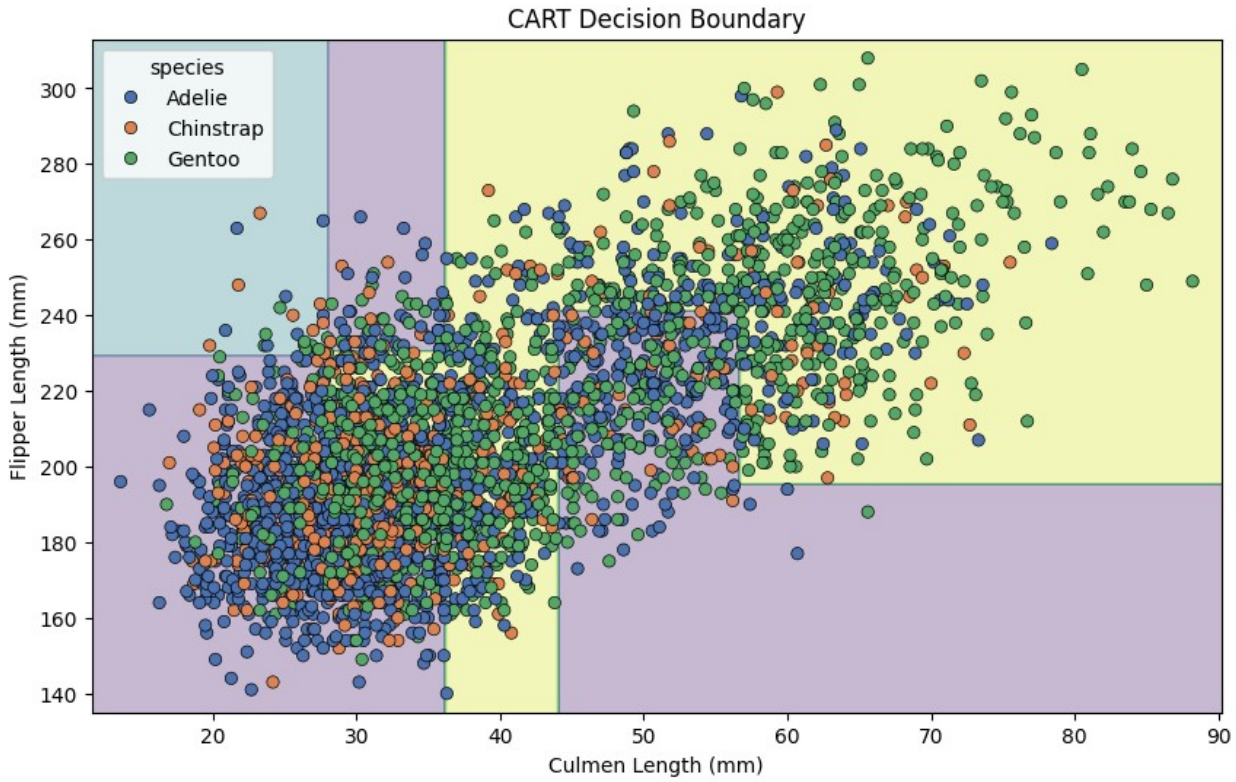
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)
```

```
grid = pd.DataFrame({
    "culmen_length_mm": xx.ravel(),
    "flipper_length_mm": yy.ravel()
})

Z = cart_2d.predict(grid).reshape(xx.shape)

plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap="viridis")
sns.scatterplot(
    data=df,
    x="culmen_length_mm",
    y="flipper_length_mm",
    hue="species",
    palette="deep",
    edgecolor="black"
)

plt.title("CART Decision Boundary")
plt.xlabel("Culmen Length (mm)")
plt.ylabel("Flipper Length (mm)")
plt.show()
```



Task 7: Normal Linear Classification Boundary

- Train a Logistic Regression model using the same two features as a normal linear baseline.

- Visualize its linear decision boundary.

```
from sklearn.pipeline import make_pipeline

log_model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000, random_state=42)
)

log_model.fit(X2_train, y2_train)

y_pred_log = log_model.predict(X2_test)

log_accuracy = accuracy_score(y2_test, y_pred_log)

print("Logistic Regression Accuracy:",
      round(log_accuracy * 100, 2), "%")

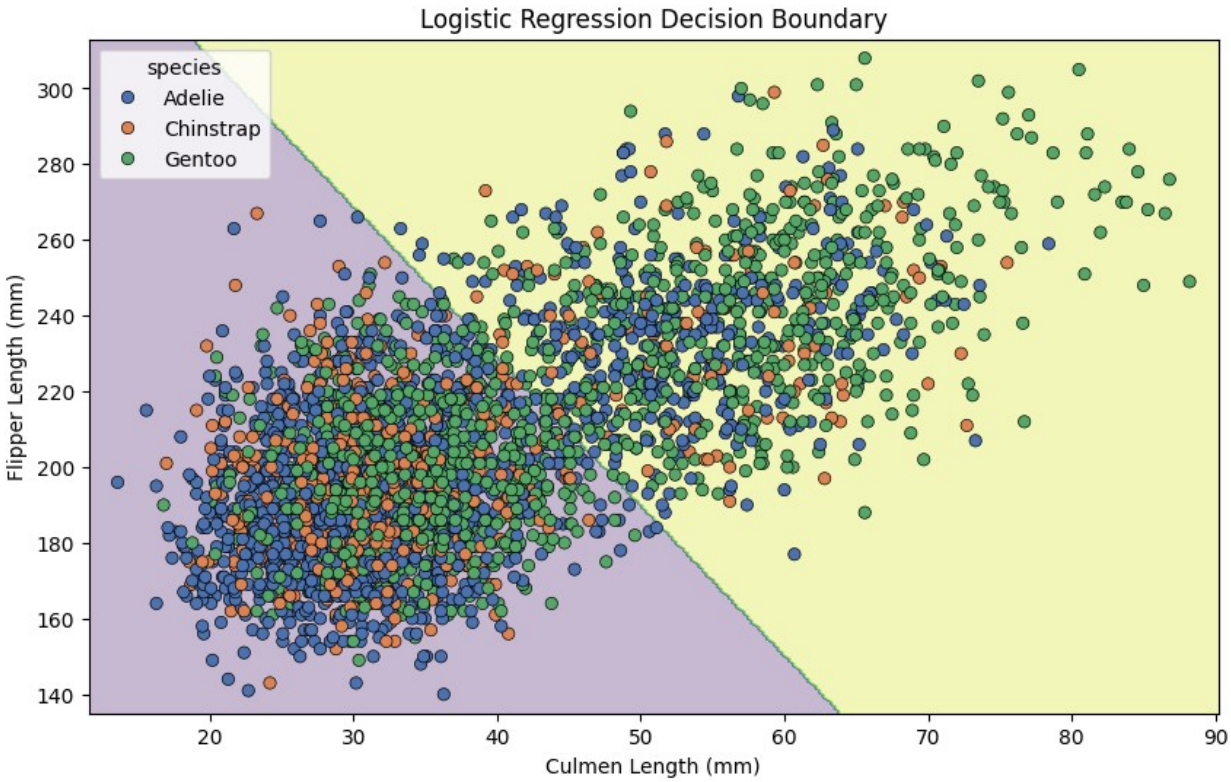
# Predict the classes on the same grid
Z_log = log_model.predict(grid).reshape(xx.shape)

plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z_log, alpha=0.3, cmap="viridis")

sns.scatterplot(
    data=df,
    x="culmen_length_mm",
    y="flipper_length_mm",
    hue="species",
    palette="deep",
    edgecolor="black"
)

plt.title("Logistic Regression Decision Boundary")
plt.xlabel("Culmen Length (mm)")
plt.ylabel("Flipper Length (mm)")
plt.show()
```

```
Logistic Regression Accuracy: 52.62 %
```



Task 8: Compare CART and Normal Linear Boundary

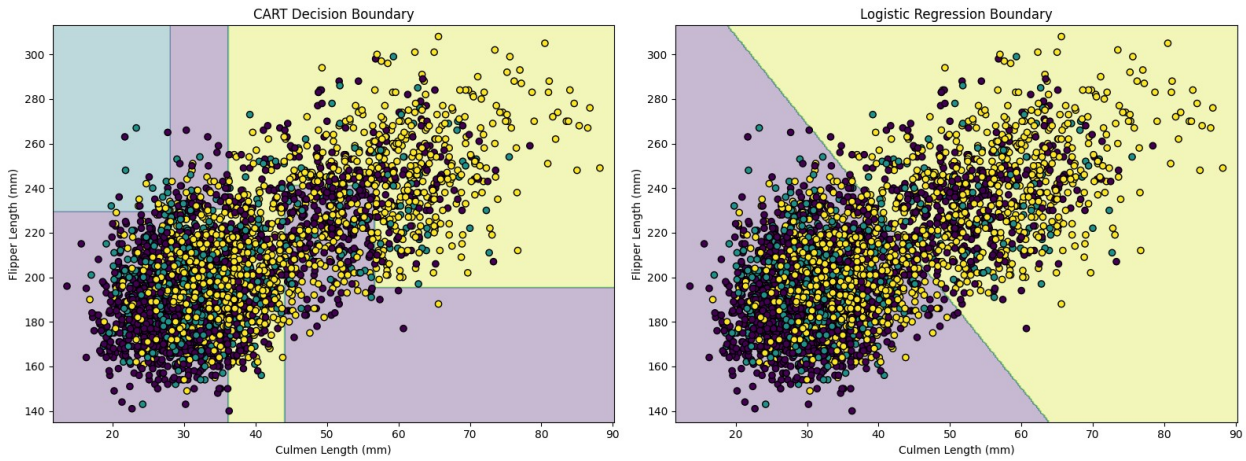
- Plot both the Decision Tree (axis-aligned non-linear) and Logistic Regression (linear) decision boundaries on side-by-side subplots. * Compare how non-linear CART boundaries differ from straight linear hyperplanes.

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CART boundary
axes[0].contourf(xx, yy, Z, alpha=0.3, cmap="viridis")
axes[0].scatter(
    X2["culmen_length_mm"],
    X2["flipper_length_mm"],
    c=y2,
    cmap="viridis",
    edgecolor="black",
    s=35
)
axes[0].set_title("CART Decision Boundary")
axes[0].set_xlabel("Culmen Length (mm)")
axes[0].set_ylabel("Flipper Length (mm)")
```

```
# Logistic Regression boundary
axes[1].contourf(xx, yy, Z_log, alpha=0.3, cmap="viridis")
axes[1].scatter(
    X2["culmen_length_mm"],
    X2["flipper_length_mm"],
    c=y2,
    cmap="viridis",
    edgecolor="black",
    s=35
)
axes[1].set_title("Logistic Regression Boundary")
axes[1].set_xlabel("Culmen Length (mm)")
axes[1].set_ylabel("Flipper Length (mm)")

plt.tight_layout()
plt.show()
```



Task 9: Compare Performance

- Compare the evaluation metrics (Accuracy, Confusion Matrix, Classification Report) of the CART Classifier vs. Logistic Regression.

```
# Train CART and Logistic Regression on the same 2D data
cart_2d.fit(X2_train, y2_train)
log_model.fit(X2_train, y2_train)

y_pred_cart_2d = cart_2d.predict(X2_test)
y_pred_log = log_model.predict(X2_test)

# Accuracy comparison
cart_2d_accuracy = accuracy_score(y2_test, y_pred_cart_2d)
log_accuracy = accuracy_score(y2_test, y_pred_log)
```

```
comparison = pd.DataFrame({
    "Model": ["CART", "Logistic Regression"],
    "Accuracy (%)": [
        round(cart_2d_accuracy * 100, 2),
        round(log_accuracy * 100, 2)
    ]
})

print(comparison)

# Plot accuracy comparison
plt.figure(figsize=(8, 5))
sns.barplot(
    data=comparison,
    x="Model",
    y="Accuracy (%)"
)
plt.title("CART vs Logistic Regression Accuracy")
plt.ylim(0, 100)
plt.show()

# Confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

ConfusionMatrixDisplay.from_predictions(
    y2_test, y_pred_cart_2d,
    labels=np.arange(len(le.classes_)),
    display_labels=le.classes_,
    cmap="Blues",
    ax=axes[0]
)
axes[0].set_title("CART Confusion Matrix")

ConfusionMatrixDisplay.from_predictions(
    y2_test, y_pred_log,
    labels=np.arange(len(le.classes_)),
    display_labels=le.classes_,
    cmap="Greens",
    ax=axes[1]
)
axes[1].set_title("Logistic Regression Confusion Matrix")

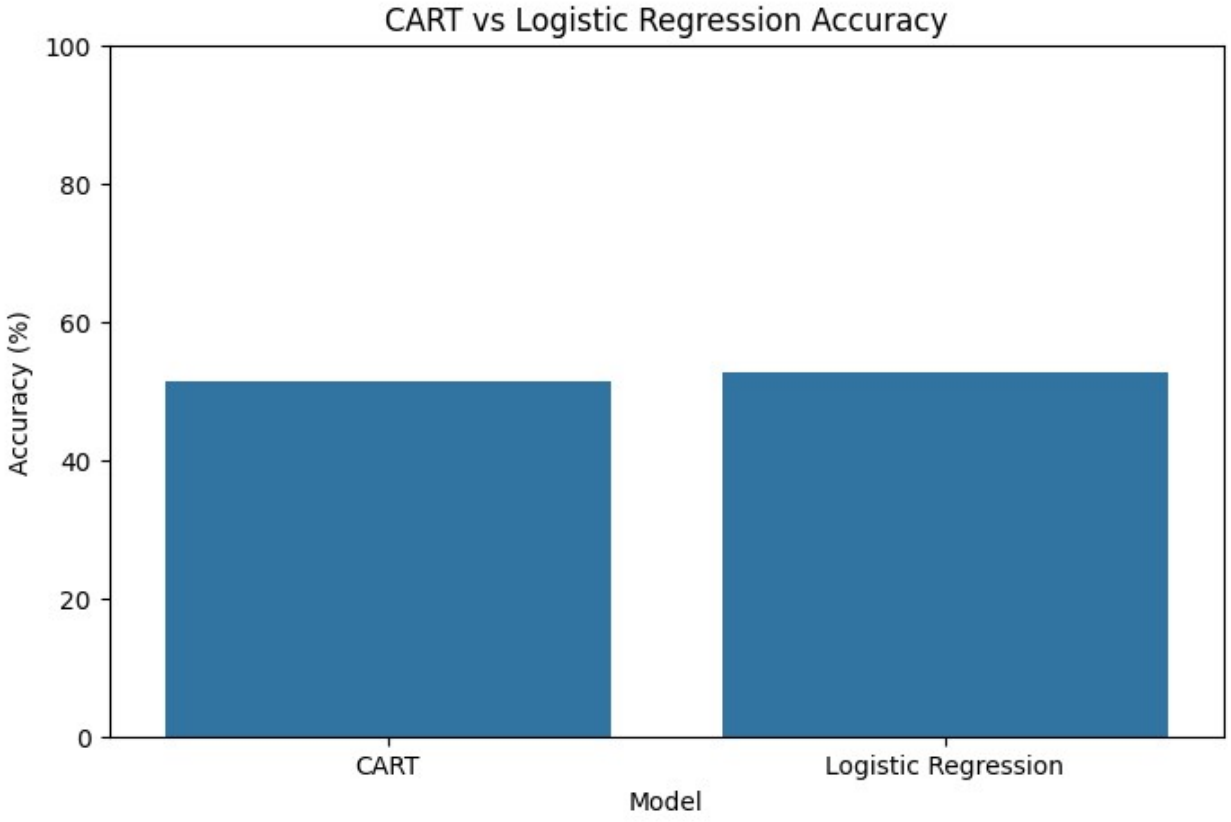
plt.tight_layout()
plt.show()

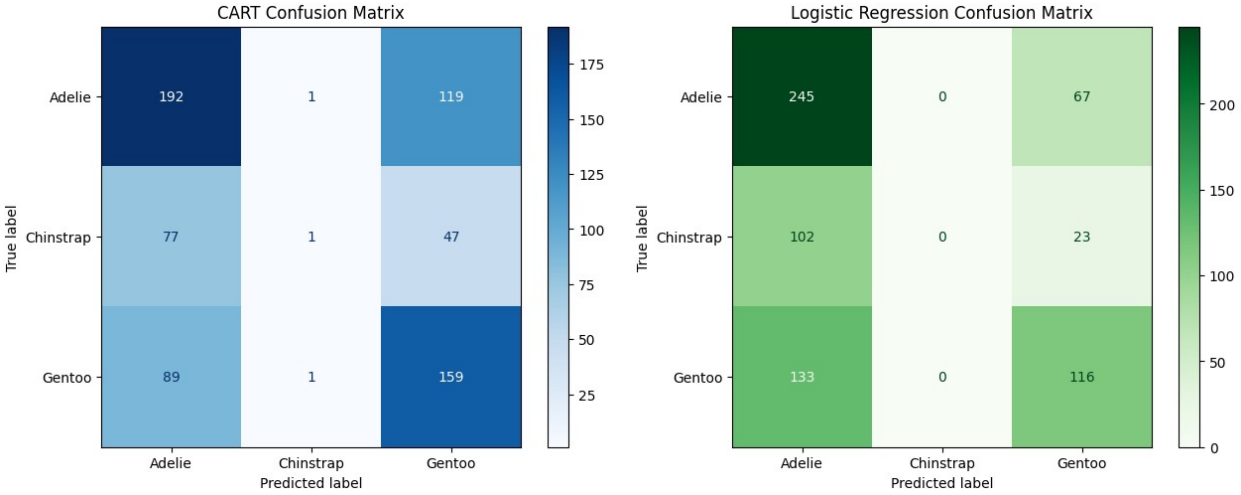
# Classification reports
print("CART Classification Report:")
print(classification_report(
    y2_test, y_pred_cart_2d,
    labels=np.arange(len(le.classes_)),
```

```
    target_names=le.classes_,
    zero_division=0
))

print("Logistic Regression Classification Report:")
print(classification_report(
    y2_test, y_pred_log,
    labels=np.arange(len(le.classes_)),
    target_names=le.classes_,
    zero_division=0
))
```

```
                 Model  Accuracy (%)
0                 CART         51.31
1  Logistic Regression         52.62
```






```
CART Classification Report:
              precision    recall  f1-score   support

      Adelie       0.54      0.62      0.57       312
   Chinstrap       0.33      0.01      0.02       125
      Gentoo       0.49      0.64      0.55       249

    accuracy                           0.51       686
   macro avg       0.45      0.42      0.38       686
weighted avg       0.48      0.51      0.46       686

Logistic Regression Classification Report:
              precision    recall  f1-score   support

      Adelie       0.51      0.79      0.62       312
   Chinstrap       0.00      0.00      0.00       125
      Gentoo       0.56      0.47      0.51       249

    accuracy                           0.53       686
   macro avg       0.36      0.42      0.38       686
weighted avg       0.44      0.53      0.47       686
```

Conclusion

The experiment successfully demonstrated the implementation of the Classification and Regression Tree (CART) algorithm using the Palmer Penguins Extended dataset. The dataset was preprocessed to handle missing values before training. The decision tree structure was visualized, showcasing the hierarchical splits based on feature thresholds. A comparison with a normal linear boundary (Logistic Regression) highlighted CART's ability to create non-linear, axis-aligned decision boundaries to separate different penguin species.